

The Memory Frontier: Teaching a Fixed-Size Matrix to Remember

What happens when you replace a growing KV cache with one Hebbian matrix that never gets bigger

DataForge 2026 · Pathway Track · Concept: Associative Memory and Fast Weights

Ask a language model something it was told ten thousand tokens ago, and it answers correctly — not because it remembers in any human sense, but because it never actually forgot. Every key and value vector from every past token is still sitting in the KV cache, waiting to be compared against the current query. That's exact, and it's simple. It is also the reason a long conversation costs more memory than a short one, and why that cost grows linearly, without bound, for as long as the conversation continues.

This project, **Memory Frontier**, is built around a narrower question than "can we make attention cheaper." It asks: if you replace the growing cache with one fixed-size matrix that is updated online as new information arrives, how much do you lose — and where, specifically, does it break?

The one-sentence, falsifiable claim

A fixed-size Hebbian synaptic memory can match KV-cache retrieval accuracy at small context sizes, but degrades through interference as more facts are written — while a KV cache stays accurate but grows without bound. The live site is built specifically so a visitor can try to break this claim themselves, not just read it.

1. The design pressure behind the idea

A standard Transformer answers a question about something said earlier by keeping every past token's key and value vector in a KV cache and comparing the current query against all of them. Fast-weight and linear-attention research reframes this: instead of storing tokens, compress the past into a fixed-size state and update that state with an online learning rule (Sun et al., 2024, arXiv:2407.04620). The open question that framing raises isn't whether a fixed-size state can work — it clearly can, at some scale — it's how its accuracy degrades as more information gets written into a container that never grows.

2. What actually changes, and the trade-off

Memory Frontier implements the simplest version of that idea on purpose: a dense Hebbian fast-weight matrix $\mathbf{W}$ ($\text{dim} \times \text{dim}$), updated on every write as

$$\mathbf{W}_t = \lambda \cdot \mathbf{W}_{t-1} + \eta \cdot (\mathbf{v} \mathbf{k}^T)$$

and read back with $\mathbf{W} \cdot \text{query}$. λ is the decay/retention factor, η is the learning rate, and $\mathbf{k} / \mathbf{v}$ are the key and value vectors for a fact. Unlike the KV cache, $\mathbf{W}$'s size never changes with sequence length — it's $O(1)$ in space for a fixed dimension, against the KV cache's $O(N)$. The cost is interference: every new outer-product write is superimposed on top of every old one in the same matrix, so old associations get

blurred or overwritten rather than cleanly evicted.

On a controlled associative-recall benchmark built for this project — random key/value vectors, no semantic redundancy either system can lean on — synaptic-memory retrieval accuracy on a fixed set of 20 tracked facts falls from 100% at 50 total facts to roughly 5–15% by 1,000–10,000 facts, while the KV cache stays at or near 100% throughout — at the cost of its own memory growing roughly 300× over that same range. These are this project's own measurements, seed-fixed and reproducible through `experiments/benchmark.py`, not an independent evaluation.

3. Walking through the artifact

The live site is organized as five linked modules, each testing one piece of the claim above rather than just illustrating it.

Memory Playground

The dimension of W , the decay rate λ , and the learning rate η are all exposed as controls. Each run writes the same controlled set of facts into a real KV cache and a real synaptic memory side by side, then queries both. Nothing is precomputed unless the backend is genuinely unreachable, and the panel says so explicitly when that happens — the accuracy and memory-footprint numbers shown are read from that run, not typed in.

Memory Recall Lab

A visitor types a few facts — "Aditya likes basketball," "Rohan likes cricket" — then injects distractor facts to simulate a long conversation, then asks a question. Facts are converted to vectors with a disclosed deterministic hash-based toy embedding, not a real language model; the site is explicit that this is good enough to test retrieval mechanics, not semantic understanding. Raising distractors from 200 toward 5,000+ on the same query is the live, hands-on version of the interference curve shown in the Playground.

Forgetting Lab

This isolates λ specifically. Higher λ keeps old synaptic strength around longer — but that also means old writes keep contributing to every later retrieval, which is exactly what causes interference. The site is careful not to oversimplify this: at $\lambda \approx 1.0$, nothing decays, so every past write keeps contributing to W at nearly full strength, which typically *increases* interference at large sequence lengths rather than fixing it, because more facts stay "loud" at once.

Connecting to Dragon Hatchling (BDH)

This is where the prototype is most careful about what it is and isn't. The site states plainly that it isolates one mechanism from a real, published architecture, and is not a reimplement of BDH.



Update rule	Dense Hebbian, single matrix per step	Sparse Hebbian writes between neuron-like units
Activation pattern	Dense, real-valued	Sparse & non-negative — ~5% of neurons active in reported runs
Connectivity	Uniform, dense	Scale-free / heavy-tailed hub structure

Per BDH's published account, its sparsity and monosemantic-synapse design are specifically aimed at reducing the interference this prototype measures in its plainest, unconstrained form (Kosowski et al., 2025, arXiv:2509.26507) — which is exactly why this prototype's accuracy degrades faster than BDH's reported long-context results. BDH-CQ extends the same family of ideas further: it stores task demonstrations in recurrent memory at inference time and reasons iteratively in a continuous latent workspace instead of a verbalized chain-of-thought, reaching 29.5% pass@2 on the public ARC-AGI-1 evaluation set at roughly \$0.0007 per task — a result independently reproduced by a team from Bielik AI and NYU (Pathway 2026, BDH-CQ, arXiv:2608.09888). BDH-CQ's contribution to this project is conceptual — adaptation living in state rather than in weights — it is not a component this prototype reuses directly.

60-Second Judge Demo

A single button runs a short benchmark sweep live against the backend and prints the exact claim being tested next to the numbers that support or challenge it, so a judge can verify the central claim without reading the rest of the site first.

4. Where this sits among recent systems

The comparison this project draws is not "which system is better" — it's where each system sits on the same trade-off curve, and what it does about the interference problem.

KV cache / full attention	Grows, $O(N)$	N/A (exact storage)	Well-established baseline
Linear attention w/ fixed state (Arora et al., 2024)	Fixed, $O(1)$	Better feature maps / hybrid tuning	Benchmarked (public)
Dragon Hatchling (Kosowski et al., 2025)	Fixed, $O(1)$, neuron-graph state	Sparse, non-negative, scale-free connectivity	Benchmarked; reported long-context results
This prototype	Fixed, $O(1)$	None — dense, unconstrained	Own controlled benchmark

None of these systems are strictly "better" in isolation: linear-attention variants trade some recall for throughput and are competitive when tuned (Arora et al., 2024, arXiv:2402.18668); BDH's sparsity is specifically motivated by reducing the interference this project measures in its plainest form (Kosowski et al., 2025). A companion line of recent work studies the same forgetting problem directly and proposes an exact auxiliary memory to fix it (Fang et al., 2025, arXiv:2607.02303) — this project's own degradation curve is best read as an illustration of the problem that motivates work like theirs, not a critique of any specific system.

5. Why the numbers on the site are real

The project brief for this track is explicit that fabricated benchmark numbers are disqualifying, so this was treated as a hard constraint rather than a nice-to-have. Every metric shown in the Playground, the Recall Lab, and the Judge Demo is computed by the FastAPI backend at request time from a real NumPy implementation of both the KV-cache baseline and the synaptic-memory update — there is no hardcoded table of "KV = 100 MB, Synaptic = 0.06 MB" sitting in the frontend. Random seeds are fixed so the same configuration reproduces the same numbers, and the site says so directly when the backend is unreachable and a chart would otherwise be misleading.

6. Limitations, stated plainly

- bullet
- This is a simplified Hebbian/fast-weight prototype **inspired by** synaptic-plasticity ideas used in BDH. It is **not** a reimplementa-tion of BDH and should not be treated as one.

- bullet The Recall Lab's free-text embedding is a deterministic hash-based toy embedding with no semantic understanding — it distinguishes strings, it does not understand language.
- bullet "Synaptic memory is $O(1)$ in space" refers specifically to the memory matrix size with respect to sequence length, for a fixed memory dimension — not to compute cost or to every aspect of the system.
- bullet Degradation curves are measured on a controlled, uncorrelated associative-recall benchmark (random unit vectors). Real-world text has structure a language model can exploit; these numbers should not be read as a prediction of real-world model behavior.
- bullet This prototype's dense, unconstrained Hebbian update is a deliberately weak baseline — it has none of the structural features (sparsity, non-negativity, monosemantic synapses) that BDH's own paper connects to lower interference. Its degradation curve is an upper bound on the problem fixed-size memory faces in the worst case, not a measurement of how BDH itself would perform on the same task — that comparison remains untested here and is the natural next experiment.

7. Reproducing the results

The benchmark sweep behind every chart on the site is a standalone script, `experiments/benchmark.py`, run with fixed seeds against the same associative-recall task described above. The full architecture, exact install and run commands, and endpoint reference for the FastAPI backend (`/experiment/run`, `/memory/write`, `/memory/read`, `/benchmark/run`, `/recall/test`) are documented in the repository README rather than repeated here, to keep this blog focused on the idea and the result rather than the setup.

Sources

Kosowski et al., 2025, *Dragon Hatchling* (arXiv:2509.26507) · Pathway 2026, BDH-CQ (arXiv:2608.09888) · Arora et al., 2024 (arXiv:2402.18668) · Sun et al., 2024 (arXiv:2407.04620) · Fang et al., 2025 (arXiv:2607.02303). Full citation list and reproduction instructions are in README.md. Live site: `web-production-c0829.up.railway.app` · Source: `github.com/aditya1085/memory-frontier`